Reviewer Pack — Minimal Alexander-Dirac Invariant and Communication Complexi…

Entry 8412084dd085 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 23:32:33 UTC

Minimal Alexander-Dirac Invariant and Communication Complexity Rank Correlation

Entry ID: `8412084dd085` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 23:32:33 UTC

1. Conjecture

Field A (mathematical branch): Alexander-Dirac Theory **Field B** (complexity object): Communication Complexity

Statement:

> For every Boolean function f, the minimal Alexander-Dirac invariant (Alex(f)) of its associated matrix representation is linearly correlated with its communication complexity rank (comm_rank(f)), such that Alex(f) = Θ(comm_rank(f)). Equivalently, for all instances with a fixed communication complexity, there exists an associated matrix whose Alexander-Dirac invariant matches the communication complexity rank within a constant factor.

Rationale (proposer's reasoning):

> Alexander-Dirac theory provides a framework to study topological properties of matrices. By connecting this with communication complexity, we may uncover new structural insights into the complexity of information distribution tasks. The conjecture suggests that there is a deep connection between the algebraic and topological aspects of Boolean functions.

Taxonomy category: `Alexander-Dirac` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ab3c5aae127220ef`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all generated Boolean functions f up to size n=40 and their associated matrices, the correlation coefficient between comm_rank(f) and Alex(f) is within a constant factor (e.g., ±1.5), with no seed producing a correlation outside this range.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        n = len(f)
        m = [[f[i * (i + 1) // 2 + j] for j in range(i + 1)] for i in range(n)]
        rank = 0
        for row in m:
            if any(row):
                rank += 1
        return rank

    def alexander_dirac_invariant(m):
        n = len(m)
        adj_matrix = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if m[i][j]:
                    for k in range(n):
                        if m[k][i] and m[k][j]:
                            adj_matrix[i][j] += 1
                            adj_matrix[j][i] += 1
        return sum(sum(row) for row in adj_matrix) / (2 * n * (n - 1))

    def gaussian_elimination(A):
        n = len(A)
```

```
    B = [row[:] for row in A]
    rank = 0
    for i in range(n):
        if B[i][i] == 0:
            for j in range(i + 1, n):
                if B[j][i] != 0:
                    B[i], B[j] = B[j], B[i]
                    break
        if B[i][i] != 0:
            rank += 1
            for j in range(n):
                B[i][j] /= B[i][i]
            for k in range(n):
                if k != i and B[k][i] != 0:
                    for j in range(n):
                        B[k][j] -= B[k][i] * B[i][j]
    return rank

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        det += (-1) ** j * matrix[0][j] * determinant(submatrix)
    return det

def is_singular(matrix):
    return determinant(matrix) == 0

n_values = [5, 10, 15, 20, 30, 40]
instances_tested = 0
total_correlation = Fraction(0)
max_n = 0

for n in n_values:
    for _ in range(5):
        f = generate_random_boolean_function(n)
        comm_rank = communication_complexity_rank(f)
        matrix = [[f[i * (i + 1) // 2 + j] for j in range(i + 1)] for i in range(n)]
        if is_singular(matrix):
            continue
        adj_matrix = gaussian_elimination(matrix)
        alex_invariant = alexander_dirac_invariant(adj_matrix)
        instances_tested += 1
        max_n = max(max_n, n)
```

```
            total_correlation += Fraction(alex_invariant * comm_rank)

    if instances_tested < 30:
        return {
            "metric_name": "correlation",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": max_n,
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    avg_correlation = total_correlation / instances_tested
    return {
        "metric_name": "correlation",
        "metric_value": float(avg_correlation),
        "instances_tested": instances_tested,
        "n_max": max_n,
        "conjecture_holds": abs(avg_correlation) >= 1.5,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_673d40f2.py", line 131, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_673d40f2.py", line 95, in run_trial
```

```
    comm_rank = communication_complexity_rank(f)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_673d40f2.py", line 26, in communication
    m = [[f[i * (i + 1) // 2 + j] for j in range(i + 1)] for i in range(n)]
          ~^^^^^^^^^^^^^^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to provide a correlation coefficient for the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14094
2	preregistration	ollama_remote	glm4:latest	0	0	9855
3	novelty	ollama_remote	glm4:latest	0	0	8411
4	novelty	ollama_remote	glm4:latest	0	0	8375
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30293
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18794
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28216
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22691
9	judge	ollama_remote	glm4:latest	0	0	9292

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 150021 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/8412084dd085.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8412084dd085.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/8412084dd085.tar.gz` (if generated)